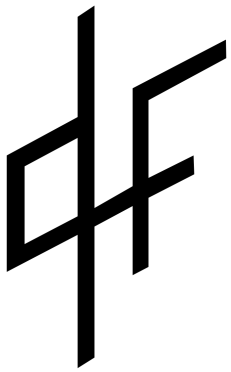
RAPPORT DE SOUTENANCE DU PROJET OCR

par la team qlf



Alexandre Pern
Eliot Soumeillan
Jean-Philippe Vonthron
Mathis Lafage

Sommaire

I. Présentation des membres..... 3
a. Alexandre Pern..... 3
b. Eliot Soumeillan..... 3
c. Jean-Philippe Vonthron..... 3
d. Mathis Lafage.....3

II. Planification et répartition des tâches..... 4

III. État d'avancement du projet..... 5
a. Le réseau de neurone XNOR.....5
b. Prétraitement de l'image..... 6
c. Rotation de l'image.....7
d. Découpage de l'image..... 8
e. Interface utilisateur..... 11
f. Solveur..... 12

IV. Conclusion..... 13

V. Bibliographie..... 15

I. Présentation des membres

a. Alexandre Pern

Je m'appelle Alexandre Pern et je suis un membre du groupe QLF pour le projet OCR. Je suis chargé de développer le **réseau de neurones** permettant la reconnaissance intelligente des caractères de l'image d'entrée, et le **solveur** de grille.

b. Eliot Soumeillan

Je me nomme Eliot Soumeillan. Dans le cadre du projet OCR du groupe QLF, j'ai pris en charge le développement des modules dédiés à la **détection** de la grille ainsi qu'à la détection et à la reconnaissance des mots et des lettres sur des images.

c. Jean-Philippe Vonthron

Je m'appelle Jean-Philippe Vonthron, je suis un membre du groupe QLF pour le projet OCR. Mon rôle dans ce projet est le développement de la **rotation** d'image ainsi que de l'**interface utilisateur**.

d. Mathis Lafage

Je m'appelle Mathis Lafage et je fais partie du groupe de projet QLF. Durant ce semestre, je suis chargé du **prétraitement** de l'image. Il s'agit d'une tâche cruciale pour le bon fonctionnement de l'OCR.

II. Planification et répartition des tâches

Tout d'abord, nous avons séparé le projet en plusieurs tâches distinctes. Le choix de ces tâches a été très judicieux car il nous a permis de faire le projet en le divisant en plusieurs sous-programmes, où chaque membre du groupe est dédié à un seul sous-programme. Une fois que tous les sous-programmes seront achevés, nous pourrons développer un programme principal qui mettra tout bout à bout et nous aurons ainsi notre solveur de mots cachés fonctionnel. Concernant la planification de toutes ces tâches, nous nous sommes tout d'abord référé aux prérequis pour la première soutenance, puis nous avons évalué la difficulté et la priorité de chaque tâche, pour déterminer par quoi commencer et où aboutir.

Tâches	Alexandre	Eliot	Jean-Philippe	Mathis
	Imagerie			
Prétraitement de l'image				X
Rotation de l'image			X	
Détection de la grille et de la liste de mots		X		
Détection des lettres et des mots		X		
	Réseau de neurone			
Réseau de neurones	X			
	Autres			
Interface Utilisateur			X	
Solveur	X			

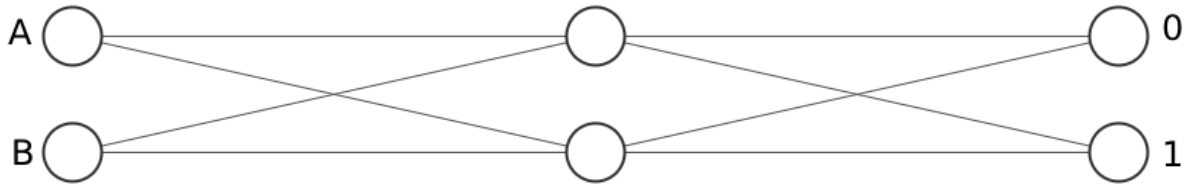
Nous avons décidé de commencer par le prétraitement de l'image car c'est un prérequis pour d'autres tâches et c'est aussi le traitement au tout début de la *pipeline OCR*. En effet, pour découper l'image, nous sommes obligés de travailler sur une image binarisée, sans bruit etc. Nous avons bien avancé sur le réseau de neurones car c'est le cœur du projet et car il s'agit d'une notion complexe à comprendre et à implémenter. De plus, le découpage de l'image a aussi été une priorité durant le développement et la conception du projet car nous considérons que l'une des tâches les plus complexes et qu'elle nécessite plus de temps que d'autres.

III. État d'avancement du projet

Concernant l'avancement du projet, nous considérons ne pas avoir de retard sur les critères attendus pour la deuxième soutenance. Nous avons même pris de l'avance sur le réseau de neurones en ayant bien avancé le réseau principal reconnaissant les lettres.

a. Le réseau de neurones XNOR

Il était demandé de faire une preuve de concept d'un réseau de neurones en réalisant un réseau capable d'apprendre la **fonction logique XNOR**. Nous avons alors réalisé un mini-réseau de neurones contenant 3 couches : une couche d'entrée, une couche cachée et une couche de sortie. Pour rendre le réseau uniforme, nous avons mis 2 neurones pour la couche cachée et pour avoir ainsi 2 neurones sur chaque couche. Nous pouvons donc représenter le réseau de neurones comme ceci :



Pour entraîner notre réseau de neurones à résoudre la fonction logique XNOR, nous avons opté pour une fonction d'activation sigmoïde sur les couches, afin d'obtenir des valeurs comprises entre 0 et 1. Afin d'ajuster les poids du réseau, nous avons utilisé la **descente de gradient** couplée à la **rétropropagation**. Concrètement, nous

avons d'abord défini une fonction de perte quadratique (MSE) qui mesure l'écart entre les prédictions du réseau et les sorties attendues. Ensuite, lors de la rétropropagation, nous avons calculé les gradients de cette perte par rapport à chaque poids en appliquant la **règle de la chaîne**, ce qui nous a permis de propager l'erreur depuis la sortie vers les couches précédentes. Enfin, nous avons mis à jour les poids dans le sens opposé au gradient (avec un taux d'apprentissage de 0,5) pour minimiser progressivement l'erreur.
Nous aurions pu faire une couche de sortie avec une seule neurone et seulement regarder la valeur de ce neurone mais nous avons préféré faire deux neurones pour chaque sortie. En effet, ce modèle est réutilisable pour la reconnaissance des lettres, où chaque neurone de sortie correspond à une lettre de l'alphabet.

```
Epoch 10000/10000  |  Loss: 0.000307  |  Acc: 100.00%
Neuron of 0 : 0.983371
Neuron of 1 : 0.016592
1 nxor 0 = 0

Neuron of 0 : 0.983357
Neuron of 1 : 0.016606
0 nxor 1 = 0

Neuron of 0 : 0.016083
Neuron of 1 : 0.983953
0 nxor 0 = 1

Neuron of 0 : 0.020477
Neuron of 1 : 0.979566
1 nxor 1 = 1

Accuracy : 100.00%
```

Voici un test de la table de vérité de l'opération logique XNOR après un entraînement du réseau. On peut observer une activation notable du neurone correspondant au bon résultat de chaque opération.

b. Prétraitement de l'image

Lorsqu'on prétraite une image pour un système de reconnaissance visuelle, notre objectif est simple : nettoyer et simplifier l'image pour en extraire l'essentiel. D'abord, nous convertissons l'image en **niveaux de gris**, ce qui permet de réduire les informations superflues tout en conservant les contrastes nécessaires à l'analyse. Ensuite, nous appliquons un **filtre médian** qui élimine les imperfections, comme les pixels isolés ou les parasites, sans altérer les contours importants. Puis, grâce à une méthode de **seuillage Otsu**, nous déterminons le meilleur seuil pour séparer clairement le noir du blanc. Enfin, nous **binarisons** l'image pour obtenir une version noire et

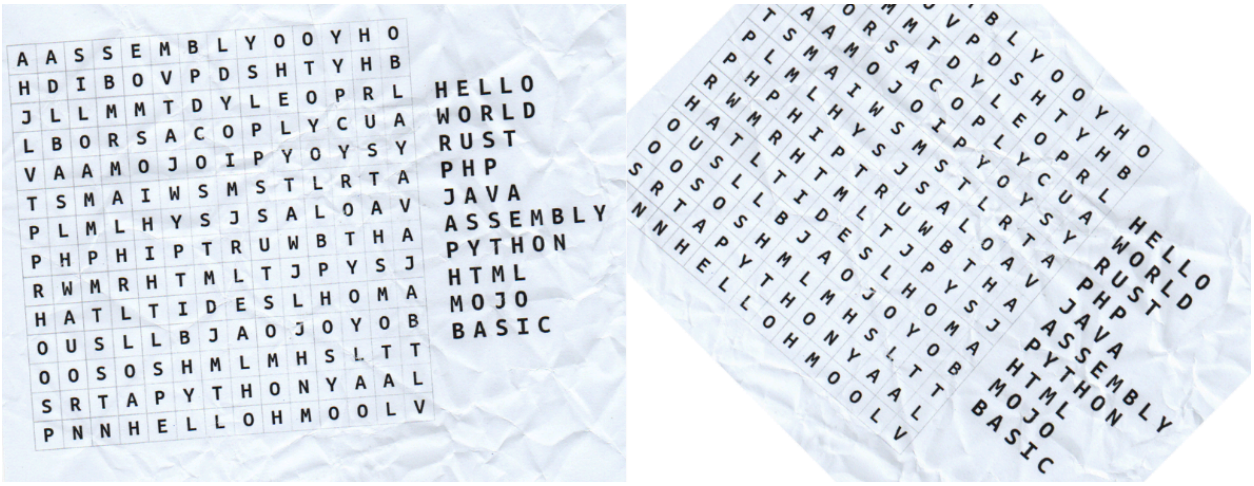
blanche, idéale pour une reconnaissance précise. Ce processus garantit que l'image finale soit propre, contrastée et prête à être analysée par nos algorithmes.





c. Rotation de l'image

Lorsqu'on applique une **rotation** à une image dans le cadre d'un traitement visuel, notre objectif est d'ajuster son orientation sans altérer sa qualité ni perdre d'informations importantes. Tout d'abord, nous calculons l'angle de rotation souhaité, puis déterminons la position de chaque pixel dans la nouvelle image à partir de sa position d'origine. Pour garantir une transformation fluide et conserver la précision des détails, nous utilisons une **interpolation bilinéaire** : cette méthode estime la couleur d'un pixel en combinant les valeurs de ses voisins les plus proches. Ainsi, les transitions restent douces et les contours nets, même après rotation. Une fois la transformation terminée, l'image pivotée est sauvegardée sous la forme d'un nouveau fichier de sortie, assurant la conservation du résultat tout en laissant intacte l'image d'origine. Cette étape finale permet de disposer d'une version corrigée, proprement orientée et prête à être utilisée pour l'analyse.



Problèmes rencontrés:

Au cours du développement de la fonction de rotation d'image, plusieurs difficultés techniques ont été rencontrées et progressivement résolues. Le premier problème concernait la dépendance à la bibliothèque *libpng*, initialement utilisée pour manipuler les images. Comme le projet devait fonctionner sans cette dépendance car non disponible sur les pc de l'école, la solution retenue a été d'utiliser la bibliothèque GdkPixbuf, intégrée à GTK, qui permet de charger, modifier et sauvegarder des images de manière plus légère et directe.

Ensuite, les premières versions de la rotation entraînaient une perte de pixels, notamment au niveau des bords de l'image. Ce phénomène provient du fait que la rotation s'effectuait à l'intérieur du cadre d'origine, ce qui coupait les coins après transformation. Pour corriger cela, une première approche a consisté à agrandir la surface de destination afin de contenir l'ensemble de l'image tournée. Cependant, cette solution provoquait un autre effet indésirable : la taille de l'image variait après rotation, ce qui n'était pas acceptable pour l'application.

Une version ultérieure a donc été modifiée pour conserver la taille originale du canevas tout en remplissant les zones vides avec un fond blanc. Toutefois, cette méthode a provoqué de nouveaux problèmes, tels que des "trous" ou des effets d'escalier. Ces imperfections provenaient de l'utilisation d'un simple arrondi lors du calcul des coordonnées des pixels. Pour obtenir un rendu plus concluant, une interpolation bilinéaire a été implémentée. Cette technique combine les couleurs des quatre pixels les plus proches afin de calculer la valeur du pixel tourné, supprimant ainsi les pertes visuelles.

Un autre problème est apparu lorsque l'utilisateur effectue plusieurs rotations successives : l'image se dégradait progressivement. Cette perte de qualité provenait du fait que chaque rotation était appliquée sur le résultat déjà tourné, accumulant les

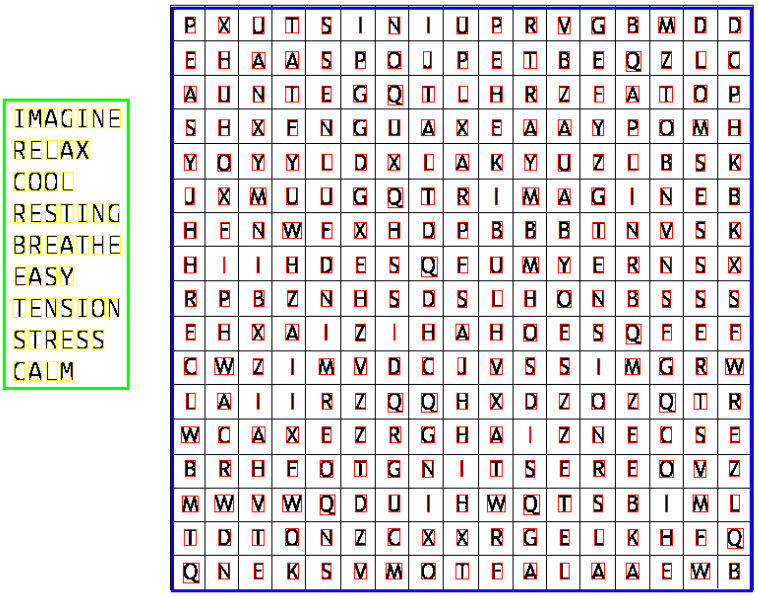
erreurs d'arrondi et les interpolations. La solution a consisté à conserver une copie de l'image d'origine, de sorte que chaque rotation soit recalculée directement à partir de cette image source, garantissant ainsi une qualité constante.

De plus, il a fallu stabiliser le centre de rotation, qui pouvait légèrement se décaler entre deux transformations. Le calcul du pivot a donc été fixé sur le centre de l'image originale, permettant une rotation centrée et une taille inchangée.

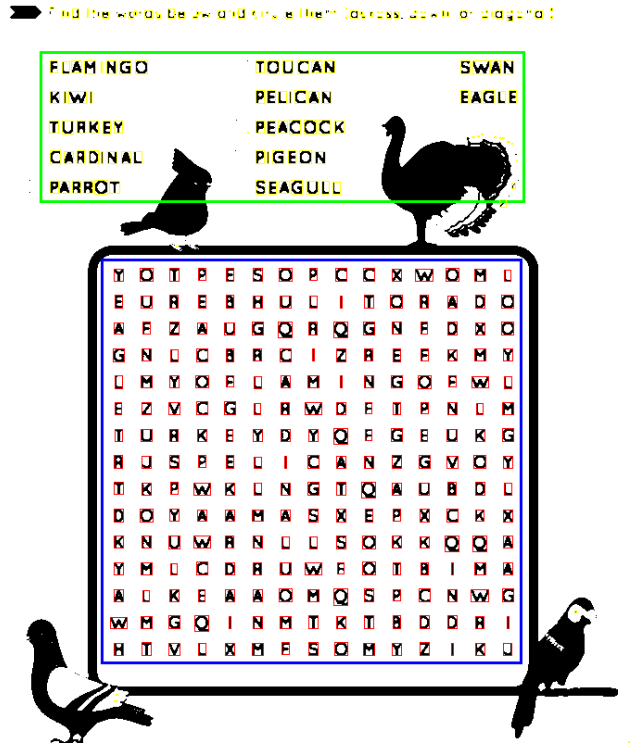
Ainsi, à travers ces différentes étapes, la fonction de rotation a été progressivement améliorée jusqu'à obtenir un résultat stable, précis et visuellement sans perte.

d. Découpage de l'image

Pour clarifier la distinction entre la zone "grille" et la zone "liste de mots", nous partons d'une lecture globale de la page puis affinons par contrastes de régularité. D'abord, nous analysons la structure spatiale: la grille se manifeste par un maillage très régulier de lignes et de colonnes, avec des espacements quasi constants; la liste, à l'inverse, présente des colonnes plus lâches, des lignes parfois incomplètes et des variations d'espacement. Cette différence de "rythme" géométrique guide toute la séparation.



Concrètement, nous regroupons les lettres selon leur proximité naturelle pour faire émerger des ensembles compacts, puis nous qualifions ces ensembles par des métriques de régularité: stabilité des lignes (espacements verticaux homogènes), présence de colonnes cohérentes (pics stables dans l'axe horizontal), compacité et densité interne. Un ensemble marquant des lignes et colonnes nettes, avec une variance d'espacement faible, sera identifié comme "grille"; un ensemble au contraire moins dense, aux colonnes irrégulières et aux espacements plus variables, sera identifié comme "liste". Cette lecture se fait à l'échelle locale (à l'intérieur de chaque zone) et à l'échelle globale (comment ces zones s'agencent entre elles). Pour réaliser cela nous avons utilisé la méthode **K-means** mais celle-ci n'était pas tout à fait adaptée à notre situation donc nous nous sommes penchés sur la méthode **DBscan** qui nous a apporté des solutions.

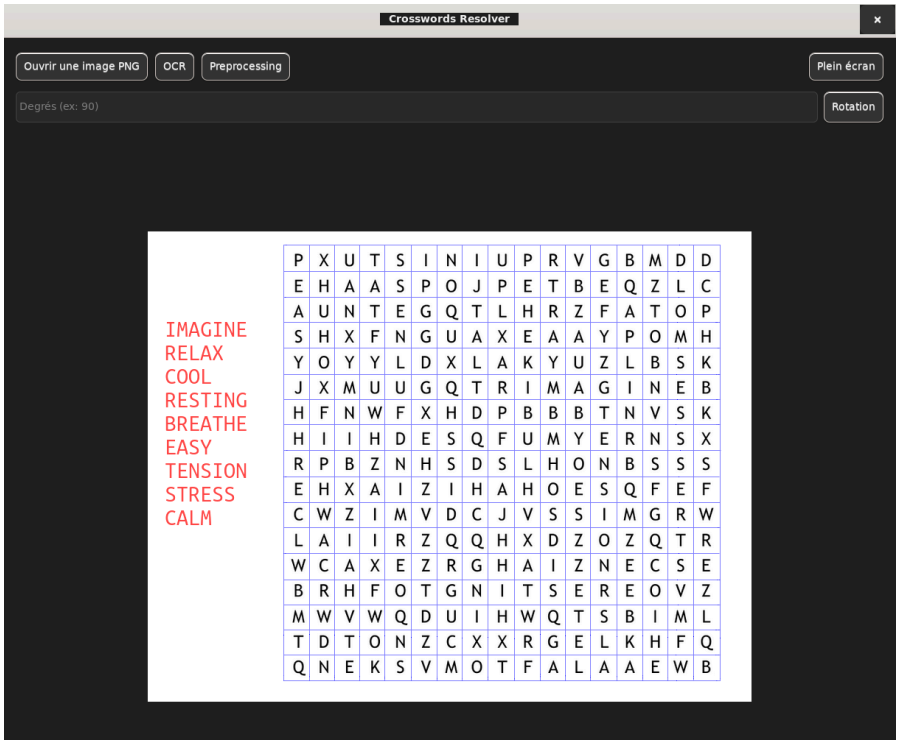


Afin de préserver les lettres fines et mieux gérer les cas limites, nous avons adapté le filtre de préparation d'image pour qu'il distingue plus efficacement les lettres des autres éléments graphiques (traits décoratifs, bordures, illustrations). Cette adaptation vise à conserver les détails utiles tout en réduisant les artefacts qui perturbent la lecture. Après séparation, un léger post-traitement typographique consolide les unités de lecture: recoller des "l" ou des traits interrompus, séparer des tirets longs fusionnés, harmoniser l'alignement vertical. Dans les rares situations

ambiguës — par exemple une liste très bien alignée ou une grille légèrement déformée — un arbitrage final s'appuie sur un petit ensemble de descripteurs de régularité, de densité et d'alignement pour confirmer l'étiquette "grille" ou "liste". Cette démarche, centrée sur la régularité géométrique plutôt que sur des seuils fixes, offre une séparation visuelle plus robuste, lisible et fidèle à la structure réelle du document. Le filtre reste toutefois perfectible et fera l'objet d'affinements supplémentaires pour améliorer encore la distinction lettres/éléments non textuels.

e. Interface utilisateur

L'interface graphique a été conçue avec **GTK 3** afin d'offrir une utilisation simple et intuitive du programme. Elle permet de charger une image, d'appliquer une rotation selon un angle défini par l'utilisateur et d'afficher instantanément le résultat. L'interface comprend plusieurs boutons : Ouvrir une image, Rotation, Preprocessing, OCR et Plein écran, regroupés dans une disposition claire. Un champ de saisie permet d'entrer la valeur de l'angle de rotation, tandis qu'une zone centrale affiche l'image avant et après transformation. Un thème sombre a été appliqué pour améliorer le confort visuel et donner une apparence moderne à l'application. La rotation est effectuée directement au sein de l'interface, à l'aide du programme externe de rotation mentionné précédemment. L'image transformée est également enregistrée automatiquement sous le nom "*output.bmp"*. Cette interface rend ainsi le traitement d'images plus accessible et visuel. Cependant les boutons OCR et Preprocessing ne sont pour l'instant pas liés à leur programme donc sont hors service pour l'instant.



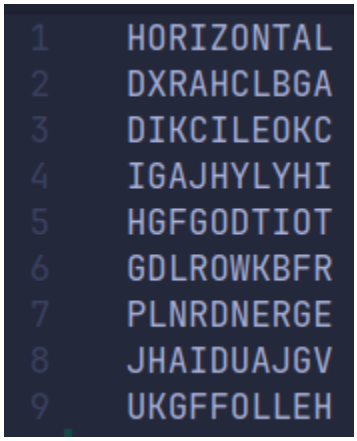
Problème rencontré:

Le bouton Plein écran rencontre des problèmes avec l'image de test "test_2_image_2" à cause de son format, problème qui n'est pas encore fixé pour le moment.

f. Solveur

Il nous était demandé de réaliser un **solveur en ligne de commande** qui renvoie les coordonnées de départ et de fin dans une grille d'un mot passé en paramètre. Pour cela, nous avons d'abord mis chaque lettre du fichier texte passé en paramètre dans une matrice de deux dimensions. Ensuite, nous parcourons cette matrice jusqu'à trouver la première lettre du mot, puis nous regardons dans les huit directions (droite, gauche, diagonales etc.) si le mot est présent. Si nous trouvons le mot, nous renvoyons les coordonnées de départ et de fin du mot.





IV. Conclusion.

Tâches	Soutenance 1	Soutenance final
	Imagerie	
Prétraitement de l'image	50%	100%
Rotation de l'image	50%	100%
Détection de la grille et de la liste de mots	100%	100%
Détection des lettres et des mots	75%	100%
	Réseau de neurones	
Réseau de neurones	60%	100%

	Autres	
Interface Utilisateur	40%	100%
Solveur	100%	100%

Le **projet OCR** de la team QLF a atteint ses objectifs majeurs et présente une chaîne globale cohérente, du prétraitement au solveur. Côté imagerie, la conversion en niveaux de gris, le filtrage et le seuillage Otsu fournissent une base propre et contrastée. La rotation d'image a été stabilisée grâce à l'interpolation bilinéaire, au recentrage sur le pivot géométrique et au recalcul systématique depuis l'original, supprimant pertes de pixels et dégradations cumulatives. La séparation "grille"/"liste de mots" s'appuie sur des critères visuels de régularité, densité et alignement, ce qui rend la découpe robuste sur des cas variés. Le réseau de neurones est opérationnel, avec une preuve de concept aboutie d'un mini réseau XNOR. Le solveur en ligne de commande fonctionne et retourne correctement les coordonnées des mots recherchés. L'interface GTK offre un parcours clair (chargement, rotation, affichage), même si certains boutons (OCR, Preprocessing) restent à lier et que le mode "Plein écran" présente encore un cas limite à corriger.

Les principaux enjeux rencontrés ont été traités par des choix techniques simples et efficaces, privilégiant la qualité visuelle et la stabilité.

En synthèse, le projet est mature: les briques critiques sont en place, les difficultés clés ont été résolues et nous sommes confiants pour la poursuite de la conception de ce projet. Ces ultimes itérations permettront de livrer un solveur de mots cachés fiable, lisible et exploitable de bout en bout.

V. Bibliographie

Comment marche un réseau de neurones ? : **3Blue1Brown sur YouTube**

Livre de Michel Nielsen : **https://neuralnetworksanddeeplearning.com/**

Comment faire une Interface avec GTK3: **aducator sur YouTube**

Les méthodes K-means et DBscan: **wikipédia** (voir leur fonctionnement)